Chapter 9. Deploying Java in the Cloud

In Chapter 8, we covered the foundational aspects of the cloud stack. In this chapter, we will take this topic further and look at the practical aspects of deploying Java processes on cloud native platforms.

We will begin by covering working locally with containers and understanding some of the basics of how containers interact when deployed. The interaction and details of how things are deployed will lead into looking at how container orchestration works and what you need to be aware of.

One enormously useful aspect of cloud native platforms is the access to ephemeral compute and the ability to scale—but this needs to be *coordinated* to be useful. With these basics in place, you will learn about options for release and deployment patterns. Deployment techniques are incredibly helpful when rolling out change to your JVM-based processes quickly, whilst still mitigating the risk of bugs.

If you are a developer, you might be wondering if deployment is really an important aspect for you to consider. Historically, you may have built software and handed it over to an operations team to run. However, one of the major changes with cloud native development is that the lines have blurred between operations and development, hence the term "DevOps."

For example, it is much simpler to create consistent environments for production and nonproduction systems. As a result, many teams are choosing to operate as "build and run" teams, finding a balance between building and supporting services. Building and operating as a single team can result in improved efficiency (or "velocity"). Efficiency gains arise from fewer miscommunications and frustrations or errors from the dev to ops handover. Build and run teams develop a deep expertise and a sense of responsibility for the stack as a whole. Working in a build and run team results in an engaged team with an increased job satisfaction.

Let's start by looking how you can work with containers locally.

Working Locally with Containers

In "Images and Containers", we covered a primer for the basics of images and containers. One of the benefits of containers is creating a more representative environment at deployment time on your local machine. Containers eliminate all of the hassles that were associated with ironing out the differences between the operating system of the developer's machine and the runtime machine.

Running the following commands will build and launch `mammal_demo` from the Fighting Animals demo. However, running the `curl` statement won't actually work due to dependencies—i.e., the other services not being available:

```
git clone https://github.com/kittylyst/fighting-animals.git .
git checkout main

mvn clean package
docker build -t mammal_demo -f src/main/docker/mammal/Dockerfile .
docker run -p 8081:8081 -t mammal_demo
curl localhost:8081/getAnimal
 {"timestamp":"2024-04-29T17:18:00.170+00:00","status":500,
 "error":"Internal Server Error","path":"/getAnimal"}
```

Looking at the map of services in `MammalController` shown in the following code, there is a DNS dependency at the code level. Both the mustelid and feline services are referred to as `mustelid-service` and `feline-service` in the URL. Later in this chapter, we will demonstrate how DNS works in orchestration platforms; however, for now we need a way to recreate this locally:

```
private static final Map<String, String> SERVICES =
  Map.of(
      "mustelids", "http://mustelid-service:8084/getAnimal",
      "felines", "http://feline-service:8085/getAnimal");
```

One option is to run a Kubernetes cluster locally; another is to use Docker Compose, which is a simpler place to start.

Docker Compose

Docker Compose is a tool for defining and running multicontainer Docker applications locally during development. A `docker-compose.yml` is used to configure your application's services and define the dependencies

between them. Then, with a single command, `docker-compose up`, you can create and start all the services based upon your configuration.

The following YAML is an example `docker-compose.yml`. The Fighting Animals example has a simple setup with five services, each of which is defined in a separate Dockerfile. The `depends_on` clauses define the service topology and the name of the services (e.g., `mustelid-service`) create a lightweight DNS entry that other containers can address:

```
# Fish service
fish-service:
  image: fish_demo:latest
  ports:
    - "8083:8083"
# Mustelid service
mustelid-service:
  image: mustelid_demo:latest
  ports:
    - "8084:8084"
# Feline service
feline-service:
  image: feline_demo:latest
  ports:
    - "8085:8085"
# Mammal service
mammal-service:
  image: mammal_demo:latest
  ports:
    - "8081:8081"
  depends_on:
    - feline-service
    - mustelid-service
# Animal service
animal-service:
  image: animals_demo:latest
  ports:
    - "8080:8080"
  depends_on:
    - fish-service
    - mammal-service
```

Running `docker-compose up` launches five distinct microservices in this example, each of them listening on a different TCP port. Running `curl localhost:8081/getAnimal` will provide a response from the mammal service. The feline and mustelid dependencies are set up on a named service and can be referenced from the other containers.

It is worth noting that the `curl` command had to be on `localhost`, as outside the containers the named services are not visible. The abstraction

of the named service is useful for creating local service names; this is consistent with orchestration systems.

TIP

One challenge for developers is that there is a lot of tooling in the development loop; in our example, there is Maven, Docker, and Docker Compose.

Without changing our build and local deployment tools, it would be helpful to make changes and see this reflect in our local running environment.

Tilt

Tilt is a toolkit that nicely coordinates other tools to create a local workflow with microservices. After installation, a `Tiltfile` is created containing the recipe. The recipe contains multiple tasks required to compile, run, and deploy the example and will redeploy parts of your application as files identified by `local_resource` , as in the following `Tiltfile` example.

The `monorepo-java-compile` task rebuilds code after it is changed. Following this, `docker_build` runs across all the images and will be redeployed in the configuration set out by `docker_compose` :

```
local_resource(
  'monorepo-java-compile',
  'mvn clean package',
  deps=['src', 'pom.xml'])

docker_build(
  'animals_demo',
  '.',
  dockerfile='./src/main/docker/animal/Dockerfile')

// ... All other docker_build tasks elided

docker_compose("deploy/docker-compose.yml")
```

Figure 9-1 is an example of the Tilt UI, which provides visual feedback on the state of the build and running containers. It has some useful features in addition to seeing the current state of local deployments, such as quickly accessing the logs from a running container.

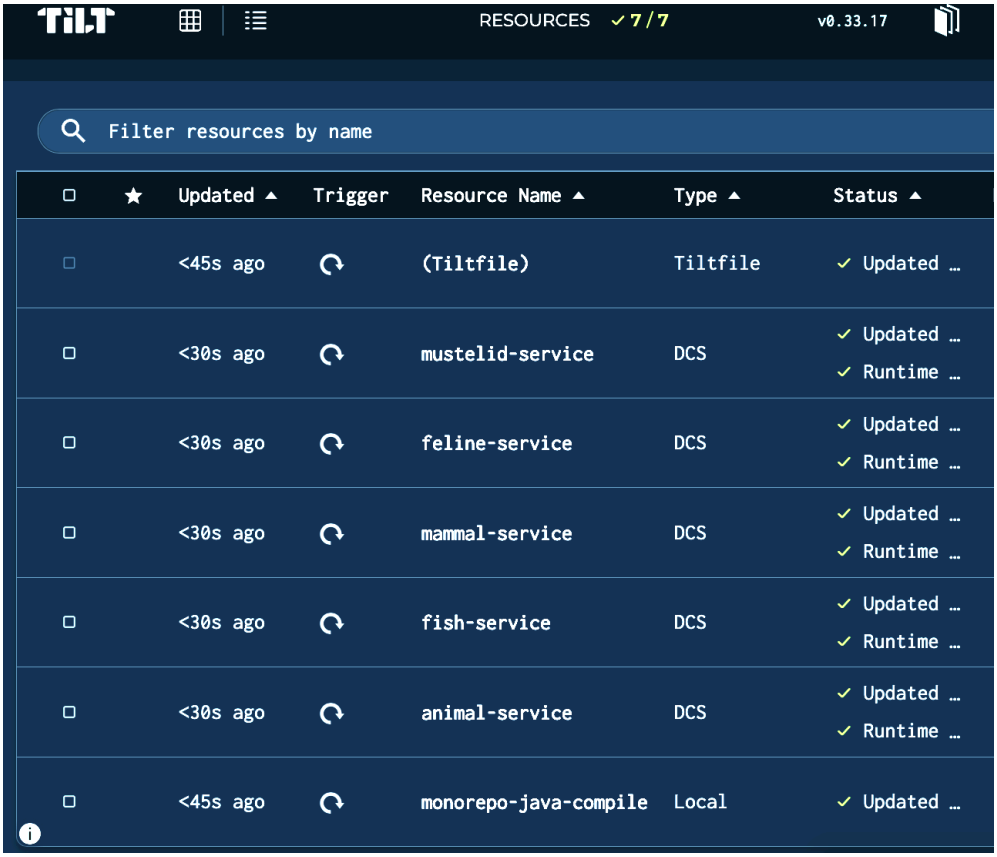

Figure 9-1. Tilt user interface

Container Orchestration

There are a variety of options for orchestrating containers; in this chapter, we will focus on Kubernetes, which is by far the most common choice.

NOTE

The name “Kubernetes” comes from Ancient Greek, and it means “helmsman” or “pilot.” Various other tools in the space play on this theme with their naming.

Cloud native container orchestration is generally implemented with two high-level fundamental components, a control plane and data plane.

The control plane is where actions are taken to adjust the state of the data plane. The data plane is where the work happens and where the five Fighting Animals services will run. From a developer perspective, thinking about where the service is running and how it is addressed is nicely abstracted into a platform concern handled by the control plane.

NOTE

Control plane operations are eventually consistent, meaning operations applied will take time to apply to the data plane. Tradeoffs are made between consistency and availability; Kubernetes, by design, aims for availability. You will learn more about distributed patterns in Chapter 14.

There are a couple of approaches to running Kubernetes locally, which will construct a local cluster. For the rest of this chapter, we will assume that you have picked one of the possible configurations—all of the instructions should work regardless of the choice you've made.

Let's take the mammal service and look at the interactions with the Kubernetes control plane to get the service running.

The key command to perform Kubernetes operations is `kubectl`, which is short for "Kubernetes Control." In practice, many developers will create Kubernetes aliases for working with `kubectl`. For example, `k` is a common alias you'll see in documentation and examples on the internet.

Deployments

A `Deployment` describes what should happen to the workloads running in the data plane. In the example deployment, the `mammal-service` is defined along with the container specification. The deployment shown in the following example is defined in a `deployment-mammal.yaml`. Running `kubectl apply -f deployment-mammal.yaml` applies our deployment in the cluster:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mammal-service
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mammal-service
  template:
    metadata:
      labels:
        app: mammal-service
    spec:
      containers:
        - name: mammaldemo
          image: mammal_demo
```

```
        ports:
          - containerPort: 8081
```

Once the `Deployment` is applied, the Kubernetes scheduler will create a Pod and orchestrate the deployment onto a node in the data plane. A Pod is one or more containers deployed together on a node in a Kubernetes cluster. In the deployment for mammal service, a single Pod will be created with a single container.

For example, in the following output, the Pod has a unique name and 1/1 containers running—meaning that the Pod has a single container running and expects to have a single container running; in other words, the Pod has reached a consistent state:

```
kubectl get Pods
NAME                                READY   STATUS    RESTARTS   AGE
mammal-service-79b4ccb9bb-4bqrj     1/1     Running   0          3m56s
```

Let's explore some of the reasons you might wish to run multiple containers within a Pod.

Sharing Within Pods

Developing with Pods creates powerful abstractions and is the heart of several common deployment architectures:

- Containers that run within a Pod have shared access to shared storage volumes and network.
- Containers deployed together in a Pod have a close coupling or a compositional dependency on each other.
- Containers deployed in a single Pod share `localhost`, making it possible for latency-sensitive services with out-of-process communication to be deployed together. This is possible because of the loopback adapter, which intercepts traffic as it travels down the network stack before it hits the physical network. This fact could be part of your design for application deployment, and it is also commonly used within infrastructure-level projects.

Egressing from one Pod to another will incur additional network latency and is something you will want to observe in the performance of your overall request flow. The impact of this will be higher if the connecting service is located on a different node in the cluster.

Service mesh is a group of CNCF projects that relies on Pods to be configured such that it can take control of network traffic and routing. A full coverage of service mesh can be found in *Mastering API Architecture* by James Gough et al.

Service mesh projects such as Istio deploy an Envoy proxy into the same Pod as a user's application container. It could be tempting to think this would require code to change to route through the proxy, which will be running on `localhost`. However, it is possible to manipulate the IP tables (i.e., routing and firewall rules) inside the Pod to alter the behavior of the local, Pod-level network. To set up the Pod in this way, an Istio-supplied `init-container` process executes setting any required state and configuration in the Pod's IP tables, and then exits.

Service mesh is used to provide additional features beyond what is provided out of the box by Kubernetes:

- The proxy can enforce that all traffic uses TLS or mTLS when traversing a cluster. This is transparent to the developer, and this is due to the intercept at the Pod level. The application connects to its local proxy, and the proxy takes on the responsibility of adding the encryption.
- As traffic is encrypted at the proxy, the proxy has access to the payload unencrypted. This enables a point of integration for telemetry collection at the network level.
- The proxy is capable of finer grain routing of traffic to target services. One example would be to prioritize traffic for human users versus traffic that is for batch or longer running processes.

Pods offer a consistent abstraction as a building block for more complex systems. With the ability to manipulate and configure shared resources consistently, Pods provide a highly flexible, consistent abstraction on top of the physical node.

Container and Pod Lifecycles

Additional nonfunctional requirements are necessary for applications running in distributed and scheduled environments.

One essential requirement is the need to provide *liveness* and *readiness* health checks. A liveness check assesses if the application is running at all, and a readiness check indicates whether the process is ready to serve requests. Accuracy of these health checks ensures that the orchestration or traffic flow systems deliver reliability and resiliency.

Note that defining readiness is down to the implementor or architect, and it should consider whether all the dependencies are in place to successfully serve a request. It is important to be able to observe both independent container health and the overall health of the system.

Pods have an execution cycle described by the current lifecycle phase, which directly taps in to the liveness and readiness checks at the container level. Pods have conditions `PodScheduled`, `ContainersReady`, `Initialized`, and `Ready`. For more information, consult the Kubernetes Documentation for Pod Lifecycle.

Ensuring that liveness and readiness checks are accurate in our application means that a Pod will not be scheduled into rotation (attached to a service) until it is in the `Ready` state. The blog post "Kubernetes Probes: Startup, Liveness, Readiness" by Levent Ogut documents this in more detail.

Pods are ephemeral, so to consistently connect to containers running inside Pods, it is necessary to introduce a capability of routing to multiple Pods across the cluster—this is where services come in.

Services

A `Service` provides an abstraction over Pods deployed in the cluster and is the basis for advertising a lightweight DNS entry and routing across the cluster.

In the following example, we deploy a Service with the name `mammal-service`. We will create a DNS entry at the cluster level—similar to that used for `docker-compose`. The selector matches our deployment; however, it is possible to specify version matches and matches on other metadata:

```
apiVersion: v1
kind: Service
metadata:
  name: mammal-service
spec:
  selector:
    app: mammal-service
  ports:
    - protocol: TCP
      port: 8081
      targetPort: 8081
```

TIP

In simple deployments, manually typing and retyping string metadata is OK, but this can get complicated quite quickly. Tools like Helm and Kustomize help address this issue by introducing data types, templates, and variables.

Running `kubectl get services` will display the Services that are running on the cluster in the default namespace as demonstrated by the following output. A Service created in this way is designed for communication within the cluster:

```
kubectl get services
NAME              TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)    AGE
kubernetes        ClusterIP   10.96.0.1        <none>        443/TCP    16d
mammal-service    ClusterIP   10.105.113.150   <none>        8081/TCP   57s
```

For clusters to accept external traffic, an external ingress point needs to be configured.

Connecting to Services on the Cluster

The Pods and Services created so far are visible only within the cluster. This is a valuable feature of cluster deployments, but in many applications it will be necessary to expose an entry point to the cluster. There are several options for this; however, a common approach is to create a `LoadBalancer` on an IP address accessible external to the cluster.

In the Fighting Animals example, only the *animal service* should be exposed outside the cluster. The other services are only referenced as internal services, which can use `Service` for internal cluster routing. In the following Service YAML, the `animal-service` is created with a `LoadBalancer` on the spec. This will indicate that an external load balancer with an IP address should be registered:

```
apiVersion: v1
kind: Service
metadata:
  name: animal-service
spec:
  type: LoadBalancer
  selector:
    app: animal-service
  ports:
    - protocol: TCP
```

```
    port: 8080
    targetPort: 8080
```

Creating this ingress point will expose an external IP address, which you can connect to from outside the cluster. Running `kubectl get services` in the following output displays the services with `ClusterIP` and the `animal-service LoadBalancer` type along with the external IP (20.108.87.2) and port (8080). You can now connect using the external IP address on `http://20.108.87.2:8080`:

```
NAME              TYPE           CLUSTER-IP       EXTERNAL-IP   PORT(S)
animal-service    LoadBalancer   10.106.223.136   20.108.87.2   8080:317
feline-service    ClusterIP      10.101.233.232   <none>        8085/TCP
fish-service      ClusterIP      10.101.40.193    <none>        8083/TCP
kubernetes        ClusterIP      10.96.0.1        <none>        443/TCP
mammal-service    ClusterIP      10.111.138.65    <none>        8081/TCP
mustelid-service  ClusterIP      10.98.142.128    <none>        8084/TCP
```

`LoadBalancer` implementations vary depending on the variant of Kubernetes you are running. For example, when deploying to a cloud provider, this will be a load balancing solution offered by the provider's network.

Next, let's look at some of the common challenges with using Kubernetes as your production environment.

Challenges with Containers and Scheduling

As with many deployment environments, it is simple to get things up and running and to work locally with Kubernetes—however, operating environments at scale can pose various challenges. The full operation of a Kubernetes cluster is outside the scope of this book, but we will cover some aspects that may surprise developers.

As the cluster increases in complexity, both in terms of number of applications and number of nodes in the cluster, it can be difficult to discover the root cause of ongoing or retrospective problems. It is also tricky to view the path of a faulty request that has executed across multiple services. It is possible to look manually at logs, but with multiple running instances, this can be time-consuming and tricky to find which Pods and containers were involved in a failing request.

It is our opinion that three-pillar observability is mandatory for operating Kubernetes workloads at scale; we will cover this further in Chapter 10.

Adopting three-pillar observability also provides monitoring of the overall cluster health, which can assist in capacity management.

Image loading is an important aspect of container scheduling. The size of the image is an important discussion for platforms, especially when considering the cost of a *cold start*, a term that describes the situation when you request a container that is not in the local container registry.

The larger the image size, the longer it will take to download and be available for the process to start. This will also place more load on your networking infrastructure, and in some cases, you will pay for this bandwidth. For longer running processes, the impact is not as significant, as the startup cost is amortized over a long-running application. However, for an application that runs for only a few seconds, the cost is extremely high.

Image size is only part of the issue because start-up times for Java can be slow under certain frameworks. This has created a myth around Java not being suitable for deployment on cloud native platforms. As we saw in "Ahead-of-Time (AOT) Compilation", there are AOT compilation techniques that make Java applications launch in ~0.029 seconds and also take up significantly less image space.

To help mitigate the impact of cold starts, Kubernetes maintains a local container registry at the node level, storing cached images. In the `Deployment` object, there is an `imagePullPolicy` that is set to determine how Kubernetes should treat the relationship between the local and remote container registries. `imagePullPolicy` can have the following values:

- `IfNotPresent` will pull an image that doesn't exist on a node. The idea is that the cost of pulling the image is paid only once. This is the default when the `:latest` tag is not used for the image.
- `Always` will always pull a newer image from the remote container registry. Using `Always` may seem inefficient; however, a check is performed to only pull new layers as required. `Always` may be required if registry tags are not immutable and the registry permits updating an existing image tag.
- `Never` will never look in the remote container registry; however, this assumes that you have configured a mechanism for loading into the container registry.

One thing to avoid is using the tag `:latest` when specifying the image in a production cluster, for two reasons: first, because it essentially sets the policy to `Always` and can cause unnecessary network traffic. Second, and more important, not using a versioned image in production can po-

tentially result in uncontrolled changes hitting your production environment.

In the worst case, this can even mean breaking changes—but more insidious is the possibility of your deployment changing underneath you without your knowledge or explicit intent. This could happen when your application is auto-scaled, introducing a partial old and new version mixture. This is a major red flag (and may even subject you to criminal liability in regulated or audited industries)—you should always know exactly what is running in production and be able to reconstruct the system state if necessary.

This might not be obvious if you are used to having Java ecosystems working from a static class path and library-based binary dependencies. Using a meaningful/ versioned tag ensures that you will get a pinned version, and `IfNotPresent` reduces the risk of pulling new images unexpectedly.

WARNING

Tags are not immutable, so investigating how your image dependencies work is an important consideration when operating your cluster.

For resiliency production, clusters should always contain multiple nodes. Node placement is an important consideration, and nodes should be positioned in different data centers or public cloud availability zones. Losing a node in Kubernetes should not be a big deal, but it requires accurate placement and enough capacity to ensure that the remaining nodes can handle the placement of containers in a partial failure.

Kubernetes will generally perform a smooth distribution of containers across the various nodes in the cluster. However, operators can also control the placement/anti-placement of containers with respect to nodes using node affinity. If you are using specialized nodes for certain workloads, performance testing will ensure that you make the most out of the resources available.

Securing the Kubernetes cluster is an important challenge that needs to be considered carefully. The Kubernetes control plane is a key target for hackers—essentially because manipulating the control plane can compromise the entire cluster. The OWASP Security Cheat Sheet is a good place to start in ensuring the security of your control plane and cluster. Ingress points must be secured at the outset, as these will often be broadly available. One of the authors ran a demo of an insecure application as part of

a workshop, and after being deployed for 15 minutes, it was under active attack.

Using `namespaces` is a large topic that is mostly out of the scope of this book, but as groups of applications grow, namespaces are a key mechanism to reduce operational risk. Namespaces are essentially a grouping of resources on a cluster, which also provides isolation and configuration that applies only within a namespace. Access to control namespaces from the control plane can be locked down, so only operators from one team can deploy resources in a team's namespace.

Kubernetes Best Practices by Brendan Burns et al. (O'Reilly, 2023) is a great resource covering these topics.

Working with Remote Containers Using "Remocal" Development

In "Working Locally with Containers", we presented options for working locally with containers. Another option is to make your local container appear as though it is part of a remote cluster. This is where *remocal development* provides the advantage of working locally with your IDE and container to use debugging and profiling tools. In complicated architectures, it has the benefit of not needing to run all services locally to fully test your application.

Telepresence is a tool that provides this capability by creating a proxy between the local machine and the cluster. You can configure which services resolve locally and which should resolve to the remote cluster.

Deployment Techniques

When working in cloud native environments, understanding the difference between *deployment* and *release* helps unlock new techniques for rolling out software change. Deployment refers to changing application components (code and/or config) or infrastructure. Release is used only when a feature or change is made available to end users. There are two main consequences of considering deployment and release as separate operations:

- Deployments can change production systems without releasing features, so deploys can be more frequent.
- Releases change user-visible behavior, but deploys may or may not.

For example, with the Fighting Animals demo, you can deploy a new version of the `fish_demo` into the production environment. From this point, we have options about how we release the new `fish_demo` into our running ecosystem. Although deployed, the new feature is not active or executed by interactions with the production system.

Several useful deployment techniques can help diagnose many of the problems we'll discuss in this section (and many others besides):

- Blue/green deployments
- Canary deployments
- Feature flags and how they can contribute to an evolutionary architecture

Blue/Green Deployments

Blue/green is one of the easier techniques to understand and provides a good starting point when looking at release separation for the first time. In most cases, it can also serve as a deployment pattern without using a fully cloud native platform. To implement it, you need a *decision point* to switch between the blue and green environments in your architecture.

A decision point is a component, for example, a load balancer, that is configured for traffic to flow to different targets. The decision to configure the load balancer for blue or green is a point-in-time decision accompanying the release step. Behind the decision point, an entire copy of your software stack is running—referred to as your *blue* environment. A second copy behind the decision point is also stood up—but this one is referred to as your *green* environment and conceptually represents the next version of your platform.

There are multiple options for how blue/green can be modeled in Kubernetes. One approach is to create new services, e.g., `fighting-animals-blue` and `fighting-animals-green`.

An `Ingress` is a Kubernetes resource, which provides an entry point but also provides a richer set of configuration options than exposing a service directly on a load balancer. You can push a configuration change to the Ingress to flip between blue and green services. Figure 9-2 highlights how this option would work with Fighting Animals.

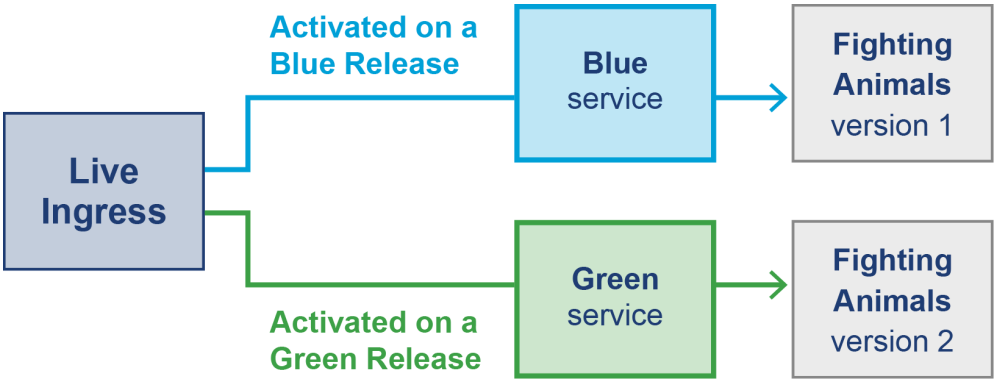

Figure 9-2. Example of a blue/green setup with Fighting Animals

During deployment of the green environment, the new changes and any regression testing can be performed in production. At the point of releasing the new change, live traffic is flipped over from blue to green. If a problem is spotted, it is a quick rollback to the previous version by updating the feature flag or which environment the gateway or ingress points to.

Moving between blue and green environments can be done as a big bang approach, and this might be a first step for developing a rollout strategy. Once all traffic is fully transferred from blue to green, the blue processes can move to standby (e.g., in case of a need to roll back). The next release will be back to the blue environment, with any new deployments created and configured against this environment. Managing ingress and services to manually move between blue and green can be a bit fiddly.

WARNING

Accessing either blue or green directly, for the purposes of regression testing, is required to verify the environment. This is necessarily different from how end users, who are oblivious to whether blue/green is active, access the environment. This can produce subtle bugs, for example, if the access is controlled by a path in the URL, and there is a bug in the path evaluation logic that doesn't manifest during regression but would manifest during go live.

In the next section on *canaries*, we will introduce Argo CD, which can also be used for blue/green deployments.

One potential downside with blue/green deployment is that you need to have all services duplicated, which can be costly. So, let's look at how *canary deployments* can provide more flexibility when replacing services without needing full replication of a blue/green environment.

Canary Deployments

Canary deployments replace running services independently and flow a small percentage of production traffic to the canary as part of a staged release sequence. The term canary originates from mining, where a canary was sent in ahead of the miners to test for any toxic gasses that might be present in the environment. The unfortunate death of the canary would prevent the onward progress of the miners (analogous to the full release).

Tools like Argo CD provide a rich set of tools for automating canary releases within Kubernetes clusters. In this section, we'll demonstrate a rollout where we set up five replicas of the `mammal_demo` image at the latest version. We will create a strategy that requires a manual intervention after the verification of the first 20% of the rollout. Following this, the rollout will introduce 20% more traffic on the new version at ten-second intervals.

To try the example yourself, you will need to install Argo CD on your cluster. You can do this by running the following commands or following the installation guide:

```
git checkout k8s-with-argo

kubectl create namespace argo-rollouts
kubectl apply -n argo-rollouts -f \
https://github.com/argoproj/argo-rollouts/releases/latest/download/instal
```

To create the strategy, you need to apply a rollout definition to the Kubernetes cluster `kubectl apply -f operations/k8s-canary-rollout-demo.yaml` . The rollout definition is picked up by ArgoCD, which you installed to the cluster.

The following YAML displays the contents of `operations/k8s-canary-rollout-demo.yaml` ; you can find this example in the Fighting Animals `k8s-with-argo` branch in `operations/k8s-canary-rollout-demo.yaml` . The rollout defines the requirement for five replicas that will initially be set to the `mammal_demo` image. The strategy is defined as canary and states that 20% should release first, and `pause: {}` instructs the rollout to wait for user input. The `pause: {duration: 10}` will not wait for user input and proceed with the next weighting after waiting for ten seconds. The starting point is five replicas running at the original version, awaiting a command to promote to a new version:

```
apiVersion: argoproj.io/v1alpha1
kind: Rollout
```

```
  metadata:
    name: rollouts-demo
  spec:
    replicas: 5
    strategy:
      canary:
        steps:
          - setWeight: 20
          - pause: {}
          - setWeight: 40
          - pause: {duration: 10}
          - setWeight: 60
          - pause: {duration: 10}
          - setWeight: 80
          - pause: {duration: 10}
    revisionHistoryLimit: 2
    selector:
      matchLabels:
        app: mammal-service
    template:
      metadata:
        labels:
          app: mammal-service
      spec:
        containers:
          - name: mammal-service
            image: mammal_demo
            ports:
              - name: http
                containerPort: 8081
                protocol: TCP
            resources:
              requests:
                memory: 32Mi
                cpu: 5m
```

Let's say that we want to release a v2 container into production. We would execute the following command:

```
kubectl argo rollouts set image rollouts-demo mammal-service=mammal_demo:v
```

This command executes the strategy defined in the previous rollout and instigates a 20% canary deployment where one container is set to `mammal_demo:v2`. Figure 9-3 shows the rollout underway using the Argo CD Dashboard; you can see this by running `kubectl argo rollouts dashboard`.

In Figure 9-3, the rollout is held at the first pause step and visually shows that one container of v2 is running. In this rollout, manually pressing

Promote will trigger the next 20% of the rollout. If a problem is found, pressing the Rollback button will remove the new version from production and perform a full rollback to the previous version.

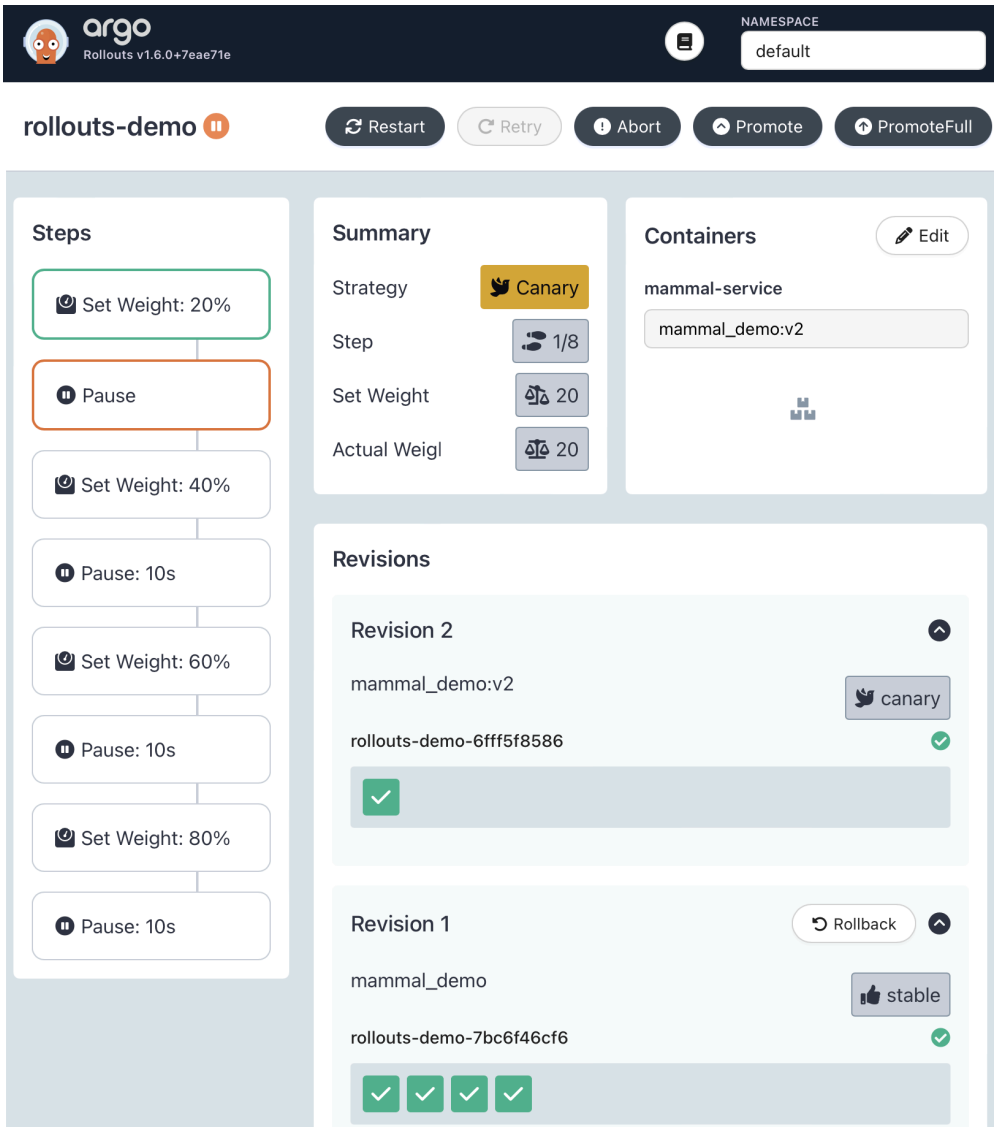

Figure 9-3. Execution of a canary release of the mammal service

In addition to manual user interaction to promote stages through the canary release, it is also possible to use signals to progress a release.

In Chapter 10, you will learn more about signals you could potentially use in monitoring for the success of a canary triggering the next stages of the release. The following is an example of an `AnalysisTemplate` that uses Prometheus metrics to look at the number of successful requests on a service.

The analysis stage would be applied as part of the rollout definition:

```
apiVersion: argoproj.io/v1alpha1
kind: AnalysisTemplate
metadata:
  name: success-rate
spec:
  args:
```

```
    - name: service-name
    - name: prometheus-port
      value: 9090
  metrics:
  - name: success-rate
    successCondition: result[0] >= 0.95
    provider:
      prometheus:
        address: "http://prometheus.example.com:{{args.prometheus-port}}"
```

With many containers running at different versions and in different locations across the cloud native stack, understanding what is going on at any given moment can be quite a daunting problem. In Chapter 10, you will learn how to handle the new level of complexity introduced by a distributed platform.

Evolutionary Architecture and Feature Flagging

As architects, the authors are often asked about moving legacy code to participate in cloud native architectures. Blue/green and canaries are really good deployment mechanisms, but how do you migrate to use them? It is unlikely that we will change complex systems all at once because this approach introduces too much risk. Evolutionary architectures do not have the same problem.

Evolutionary architectures are designed with the expectation that the system will change over time and remain open to this. The idea behind an evolutionary architecture is an execution plan for moving toward a target state architecture. *Building Evolutionary Architectures* by Neal Ford et al. is a fantastic guide to the concept.

NOTE

Evolutionary architecture is a journey, and you will learn things along the way. It's likely that your target state will shift as you learn more about technologies.

In 2016, Amazon Web Services published a "six R's" approach to cloud migration. If you are considering migrating your existing Java applications to the cloud, this represents a set of possible approaches of how you can migrate.

The six R's are:

- Retain or Revisit
- Rehost

- Replatform
- Repurchase
- Refactor/Re-architect
- Retire

It is important to know that, in some cases, it's OK to do nothing, and *Retain or Revisit* accepts that a system is too difficult to move, or perhaps adds significant value in current form.

Thinking that you need to move and change everything at once is an antipattern when adopting cloud native technologies. Only move things that are going to make a tangible difference to either your business case or improve the nonfunctional requirements in your architecture.

Rehosting takes the same platform model that you have today and moves it into the cloud architecturally unchanged in what is sometimes caled a *lift-and-shift* approach. This might seem like a pointless exercise, but there is an advantage to starting to consolidate and colocate your applications in the cloud.

This is often an initial step in an overall migration and can be seen as a useful decision point—i.e., whether to fully buy into cloud providers or understanding that a *hybrid architecture* between cloud and traditionally managed data centers makes more sense. Hybrid means that you'll use the best of cloud native platforms, but also accept that you will own or retain your own servers as well.

Replatform is somewhat similar to rehosting but involves a sliding scale of rework to adjust to the cloud environment. This might mean tweaking a few things to be able to take advantage of a slightly different solution, such as elastic managed database servers (e.g., RDS in AWS) instead of self-managed databases. At the upper end, replatforming can involve very significant changes to the application architecture and potentially overlaps with rearchitecting.

Repurchasing involves using an off-the-shelf SaaS product instead of something that you previously owned or operated.

Refactor/Re-architect is the most interesting from a technical perspective, as it provides the opportunity for adapting our software to make full use of platforms like Kubernetes.

Retiring is exactly as it sounds. Migrating might mean that some components are no longer required and can be decommissioned.

To support a migration to cloud, you will need different techniques to take advantage of new features in a controlled manner. In particular, a code-level construct is needed to provide migration at a granular level. *Feature flags* are a code-level check that use an external store of flags that can be configured to manipulate the flow of a running system, based on some condition.

This is a huge subject, and one that we will not be able to cover fully, but let's look at an example using pseudo-code from the popular Java feature-flagging tool LaunchDarkly.

In the following code example, we are performing a user-based switch on features to decide whether the new mammal service is used or an internal library is providing the functionality. The feature flag `"user.enabled.mammals"` is checked and will default to `false`:

```
LDUser user = new LDUser("authors");
boolean mammalService =
    launchDarklyClient.boolVariation("user.enabled.mammals", user, false)
if (mammalService) {
  // Retrieves the mammal from the modern environment
}
else {
  // Retrieves the mammal from the existing monolithic codebase
}
```

With this type of flagging in place, there is full flexibility to introduce and turn on/off new features as needed, effectively separating deployment and release for any application. Using feature flags in combination with canary releases and blue/green is a good approach to application migration.

NOTE

Feature flags have to be highly available in an architecture, but a sensible default is a fallback in the case of failure.

Feature flags are also commonly used as a primary mechanism for rolling out deployments. For example, in the Fighting Animals example, you could decide to use feature flags only for rolling out new changes to end users. Feature flags are at the heart of many systems that are too large to consider blue/green. For many applications that run 24-7, they are one of the few comprehensive ways of releasing change without requiring a time-consuming rollback. You must clean up feature flags and carefully

consider the naming of feature flags. Feature flags should have a defined lifetime and should not exist indefinitely in the codebase.

WARNING

Knight Capital is an extreme example where reusing feature flags and not tidying up led to half a billion dollars of electronic trading losses within a few hours. It is worth setting a policy for the naming and usage of feature flags in a system.

Finding a balance between the deployment options used for a particular stack of architecture is an important balance. Ensuring that things are not too complex, deployments are pushed quickly, and features are enabled as required in the target state.

Java-Specific Concerns

In this section, we will address some common concerns that are specific to Java deployments that applications delivered in other languages may not face. We have seen situations in production where using containers has gone wrong for a number of reasons. Let's explore some of these problems and what you should consider upfront to avoid this in your project.

Containers and GC

Both of the authors have seen situations where configurations have been set to force the JVM into a corner.

Research from Relic shows that 70%+ of Java applications are now deployed in containerized environments.[1]

By itself, this is not an issue—until we look at the distribution of CPUs commonly used in these containers. Roughly half of these containers are configured such that they appear to have only a single CPU. This is a problem because, as we discussed in "Java Implementations, Distributions, and Releases", on startup the JVM dynamically sets some properties of the VM that control runtime behavior—including the GC configuration.

Recall that the default G1 collector is partially concurrent—but to run a concurrent collector, it needs multiple CPUs. On a single-CPU machine (which is what a single-core container presents as), the JVM ergonomics will detect that G1 cannot function effectively, so the Serial and SerialOld collectors will be selected instead.

For example, setting `CPU: 1` constrains the runtime environment to have access to only one CPU. On the surface, this may seem like a reasonable idea; however, as we discussed in Chapter 5, modern garbage collectors are concurrent. The practical effect of this constraint will mean that the JVM has to run GC as a serial operation, creating larger pause times, more application interruption, and lowered throughput.

As a performance engineer, you should be aware of this effect and ensure that you deploy into single-core containers only if you are absolutely sure that there is benefit in doing so. Research by companies such as Red Hat and Microsoft indicates that for many workloads, large clusters of single-core containers are significantly less effective than smaller clusters with more CPUs per container.

In other words—it is very possible that deploying many single-core containers is wasteful, both in resources and money spent on cloud infrastructure. Therefore, the default assumption, in the absence of any other evidence, is that Java applications should be deployed in containers with two (or more) cores.

Memory challenges have caused significant issues with Java applications adopting containers—let's explore some of the effects for early adopters and developers not using the latest versions of the JVM.

Memory and OOMEs

The first issue is that older versions of the JVM historically did not observe the cgroups hints (as older versions of Java predate the development of the cgroups technology) and instead looked at the overall host machine details. What this means is that a JVM might try to use more memory than it actually has available, which would cause the operating system to kill the application. This is a major issue, because violating a cgroup limit means potential process termination, as the kernel does not fully enforce isolation using cgroups.

The cgroups functionality was backported to Java 8, but if you are using containers, you should use an up-to-date version of the JVM, as further optimizations have been added. For example, two major additions in Java 17 are cgroups v2 support and container awareness in `OperatingSystemMxBean`.

WARNING

If you run an older JVM on a machine that only supports cgroups v2, you will find yourself in a situation where the JVM is looking at host-level details rather than the container constraints.

In general, it is always worth running with the latest LTS JVM where possible (ideally 17 or 21), especially in a containerized environment. Not only are there performance improvements with each LTS version of Java but running on anything less than Java 11 could also have unusual impacts on your application on certain hardware that may not be visible in local development.

The second issue is the configuration and settings between the container and the JVM. For example, if you allocate a quota of 1 GB to a container, the JVM maximum heapsize will auto-configure to 256 MB. You need to ensure that you leave space for container internals or other processes running, but this is likely going to result in an under-utilization.

Overriding the maximum heapsize `-Xmx` and running a performance test is a good practice to ensure maximum utilization. An alternative is setting the `-XX:MaxRam` option, which declares the physical RAM available to a process, enabling the JVM to decide how to size the heap. In addition to considering the heap sizing, it's important to consider the sizing of the stack and any direct memory or off-heap allocation in your application.

The reality is that you now have a series of options to configure, constrain, and tune. There are JVM configuration options and the runtime configuration options for the orchestration layer. The configuration options cannot always be considered or tested in isolation. It is possible to run with no constraints, and in a lift-and-shift,[2] this can be a good starting point. It is unlikely that you will want this to be the target state, as this likely will not address performance and resiliency objectives.

Summary

In this chapter, we have scratched the surface of a complicated shift toward a closer relationship between development and deployment of Java applications. You have learned about tools and approaches for working locally with containers and how to create lightweight DNS entries using Docker Compose.

We have reviewed a starting point for working with Kubernetes and some key concepts for the lifecycle of Pods and containers, including some common gotchas. We have explored by example how tools like Argo CD help with the automation of releases. Finally, we have looked at approaches for separating the concept of deployment and release using blue/green, canary, and feature flags for evolutionary architecture.

Using this tooling alone without a good way to manage services in production would be very challenging. In the next chapter, we will explore observability, which should be considered mandatory for the deployment options we have shared in this chapter.

[1] This is not a perfect view of the market as a whole but is based on data from tens of millions of production JVMs.

[2] Migrating an existing architecture to another platform with minimal changes.